SPIRIT 12

Cahier des charges technique et sprints de développement

Application web d'enseignement spirituel et d'accompagnement personnalisé par agents IA

Document opérationnel pour OpenCode et guide de publication sur GitHub

Version 1.0 • 21 octobre 2026

Principe directeur : aider chaque utilisateur à transformer des enseignements en actions quotidiennes réalistes, suivies et ajustables — sans promettre une réussite garantie ni présenter les lois spirituelles comme des lois scientifiques.

1. Comment utiliser ce document avec OpenCode

Ce document décrit les fonctionnalités, l'architecture, les règles de sécurité, les sprints, les tests et le flux GitHub. Il ne faut pas demander à OpenCode de tout implémenter en une seule opération. Exécuter un sprint à la fois, vérifier les résultats, lancer les tests et committer avant de passer au suivant.

Instruction de départ à copier dans OpenCode

```
Lis intégralement le PDF SPIRIT12_Sprints_OpenCode_GitHub.pdf fourni dans le projet.
Commence par inspecter le dépôt et l'application existante. Ne remplace pas l'architecture sans raison.
Établis un état des lieux : stack, structure, scripts, tests, variables d'environnement, fonctionnalités déjà présentes et risques.
Propose un plan d'exécution aligné sur le document. N'implémente pas encore tous les sprints.
Respecte les critères d'acceptation. Ne mets jamais de secrets dans le code ni dans Git.
Avant chaque modification importante, explique les fichiers concernés. Après chaque sprint, exécute les tests disponibles, corrige les régressions et résume les changements, commandes exécutées, résultats et limites.
Travaille sur une branche Git dédiée et ne pousse rien vers GitHub sans confirmation explicite.
```

2. Vision et périmètre produit

SPIRIT 12 est une application web responsive qui aide chaque personne à étudier les 12 lois spirituelles, définir des objectifs, recevoir des missions quotidiennes personnalisées et suivre sa progression. L'IA accompagne l'utilisateur mais ne remplace ni son jugement ni les professionnels qualifiés lorsque ceux-ci sont nécessaires.

Périmètre du MVP

- Inscription, connexion, profil et préférences de l'utilisateur.
- Questionnaire de départ : objectifs, durée, temps disponible, difficultés et orientation spirituelle.
- Bibliothèque des 12 lois, leçons, exemples, exercices, citations et références sourcées.
- Création d'un parcours de 21, 30, 60, 90 ou 180 jours.
- Missions quotidiennes, validation des tâches, journal privé et bilan hebdomadaire.
- Agents IA : profil, enseignement, planification des missions, coach et contrôle de qualité.
- Tableau de bord, suivi de progression et interface d'administration minimale.
- Sécurité, confidentialité, export et suppression des données du compte.

Hors périmètre initial

- Application mobile native, réseau social public, mentorat payant et marketplace.
- Diagnostic psychologique ou médical, prédiction de destinée ou promesse de réussite.
- Agents qui agissent sans limite, modifient seuls les objectifs fondamentaux ou partagent des notes privées.
- Génération automatique non vérifiée de citations, versets ou affirmations doctrinales.

3. Règles métier fondamentales

- Chaque utilisateur ne peut lire ou modifier que ses propres données, sauf données de contenus publics et fonctions explicitement réservées à l'administration.
- Un parcours doit avoir un objectif, une date de début, une durée, des jalons et des indicateurs observables.
- L'utilisateur peut modifier, suspendre ou arrêter son parcours. Les modifications majeures sont confirmées avant application.
- Une mission comprend un titre, une consigne, une durée estimée, une loi associée, un objectif associé, un statut et une date prévue.

- Les missions proposées par l'IA sont validées par le backend selon les limites de temps, le contenu autorisé et les règles de sécurité.
- Le journal est privé par défaut. Ne pas l'injecter dans le contexte d'un agent sans besoin précis et consentement approprié.
- Ne jamais culpabiliser l'utilisateur pour une journée manquée. Proposer une reprise réaliste.
- Les lois spirituelles sont présentées comme un cadre philosophique ou traditionnel. Distinguer clairement les enseignements bibliques, les interprétations modernes et les faits vérifiables.
- Les agents ne garantissent aucun résultat. Ils peuvent proposer des exercices, des pistes de réflexion et des actions mesurables.
- Toute action persistante passe par des services backend contrôlés, jamais par une écriture directe du modèle dans la base.

4. Architecture technique proposée

Si un dépôt ou une application existe déjà, OpenCode doit d'abord l'inspecter et conserver la stack actuelle si elle est viable. Pour un nouveau projet, la stack par défaut recommandée est :

- **Frontend :** Next.js, React et TypeScript, interface responsive et accessible.
- **Backend / données :** Supabase avec PostgreSQL et authentification, ou backend équivalent si le dépôt existant le justifie.
- **IA :** fournisseur accessible via une API côté serveur. Clé secrète jamais exposée au navigateur.
- **Validation :** schémas stricts pour les sorties IA, par exemple avec Zod.
- **Automatisation :** tâches planifiées pour préparer les missions et bilans; utiliser une file de tâches si le volume le justifie.
- **Tests :** tests unitaires, tests d'intégration, tests d'accès multi-utilisateurs et tests end-to-end.
- **Déploiement :** environnement de préproduction puis production, variables d'environnement séparées.

Les agents sont des rôles logiques orchestrés par le backend. Il n'est pas nécessaire d'exécuter dix modèles séparés. Le MVP démarre avec quatre agents principaux et des règles classiques pour les tâches déterministes.

5. Modèle de données de départ

Table	Champs / rôle principaux
profiles	user_id (FK auth), display_name, timezone, preferences, created_at
goals	id, user_id, title, description, priority, target_metric, status, target_date
assessments	id, user_id, answers JSON validées, completed_at
laws	id, slug, title, summary, tradition, status, order_index
lessons	id, law_id, title, body, sources, reading_minutes, published_at
exercises	id, law_id, goal_tags, duration_minutes, difficulty, instructions, safety_notes
programs	id, user_id, goal_id, start_date, duration_days, status, version
program_steps	id, program_id, day_number, lesson_id, objective, order_index
daily_missions	id, user_id, program_id, scheduled_date, title, instructions, estimated_minutes, status
mission_events	id, mission_id, user_id, event_type, created_at
journal_entries	id, user_id, entry_date, encrypted_or_protected_content, created_at
weekly_reviews	id, user_id, program_id, week_start, summary, user_confirmed
ai_runs	id, user_id, agent_role, status, input_hash, output_schema_version, created_at
audit_events	id, actor_user_id, action, resource_type, resource_id, created_at

Adapter les noms et relations au schéma réel. Les règles Row Level Security (RLS) doivent être actives pour toutes les tables contenant des données utilisateur. Ne pas stocker de données sensibles qui ne sont pas nécessaires.

6. Spécification des agents

Agent	Mission	Autorisation
Profil	Résumer les objectifs et contraintes déclarés; demander les informations manquantes.	Lire uniquement le questionnaire et les préférences autorisées; proposer, pas modifier sans accord.
Enseignement	Expliquer une loi avec le corpus approuvé, exemples et références.	Lecture du corpus publié; citer les sources disponibles; signaler l'incertitude.
Planificateur	Proposer un parcours et des missions dans les limites du profil.	Créer une proposition structurée; backend valide durée, charge, dates et associations.
Coach	Répondre aux questions et aider à résoudre les obstacles.	Contexte minimal nécessaire; ne pas exposer le journal privé par défaut.
Progression	Analyser les événements de mission et préparer des bilans.	Lire des statistiques autorisées; distinguer faits observés et hypothèses.
Journal	Proposer des questions de réflexion et résumer à la demande.	Accès aux entrées uniquement sur action/consentement explicite de l'utilisateur.
Rappels	Choisir le moment et le ton des rappels selon les préférences.	Les envois sont exécutés par un service de notification avec limites et désinscription.
Contrôle	Contrôler format, sécurité, sources et ton des sorties IA.	Peut bloquer ou demander une régénération; ne remplace pas les contrôles backend.
Orchestrateur	Choisir quel rôle appeler et dans quel ordre.	Outils en liste blanche; journalisation; budget de tours et de tokens.
Admin contenu	Aider à rédiger et classer les contenus.	Brouillons seulement; publication réservée à un administrateur humain.

Chaque agent doit avoir : un objectif unique, un prompt système versionné, un contexte autorisé, des outils limités, un schéma de sortie validé, un budget d'exécution et une stratégie de repli en cas d'erreur.

7. Sprints détaillés

Les sprints ci-dessous sont des lots de travail. OpenCode doit inspecter le dépôt avant de choisir les fichiers ou de créer une structure. Chaque sprint doit se terminer par un rapport et un commit local proposé.

SPRINT 0 — Audit du dépôt et plan d'exécution

Objectif : Comprendre l'état réel du projet avant toute modification.

Travaux à réaliser

- Inspecter les fichiers, le framework, les scripts package, les tests, la configuration et les dépendances.
- Identifier les fonctionnalités existantes et les réutiliser; ne pas écraser un travail existant.
- Vérifier les fichiers secrets, .gitignore, licence, README et état Git.
- Créer un plan de travail par sprint avec risques, dépendances et critères de validation.

Critères d'acceptation

- Un rapport d'audit est fourni avec architecture, commandes de test et risques.
- Aucun fichier existant n'a été supprimé ou remplacé sans justification.
- Les étapes suivantes sont adaptées à la stack réellement détectée.

Tests obligatoires

- Exécuter les tests et le build existants si possible.
- Signaler les tests impossibles à lancer et pourquoi.

SPRINT 1 — Fondations, navigation et design system

Objectif : Construire une interface cohérente, accessible et responsive.

Travaux à réaliser

- Créer ou adapter la structure des pages publiques et privées.
- Définir couleurs, typographie, espacements, composants, états de chargement, erreurs et pages vides.
- Créer la navigation : Tableau de bord, Mon parcours, Les 12 lois, Journal, Progression, Profil.
- Prévoir un layout administrateur séparé et protégé.
- Ajouter les états responsive mobile, tablette et desktop.

Critères d'acceptation

- Les pages principales sont accessibles via navigation sans liens morts.
- L'interface fonctionne sur les largeurs mobile et desktop.
- Les composants ont des états loading, empty et error appropriés.
- Les bases d'accessibilité clavier et contrastes sont respectées.

Tests obligatoires

- Tests de navigation et de rendu.
- Vérification manuelle sur plusieurs largeurs.

SPRINT 2 — Authentification, profil et préférences

Objectif : Créer une identité utilisateur et isoler ses données.

Travaux à réaliser

- Implémenter inscription, connexion, déconnexion et réinitialisation de mot de passe avec le fournisseur choisi.
- Créer le profil : nom d'affichage facultatif, fuseau horaire, langue, temps disponible et préférences.
- Construire le questionnaire : objectif, durée, difficulté, disponibilité et orientation spirituelle.
- Valider les entrées côté client et côté serveur.
- Mettre en place les rôles utilisateur et administrateur.

Critères d'acceptation

- Un utilisateur peut créer son compte et reprendre une session.
- Les données de profil ne sont pas accessibles à un autre compte.
- Les erreurs d'authentification sont traitées sans révéler d'informations sensibles.
- Les préférences peuvent être modifiées et supprimées.

Tests obligatoires

- Tests d'inscription, connexion, déconnexion et réinitialisation.
- Tests d'accès croisé entre deux utilisateurs.

Notes d'implémentation

- Ne jamais committer de clés privées, mots de passe ou tokens.
- Appliquer le principe de moindre privilège.

SPRINT 3 — Base de données, RLS et migrations

Objectif : Créer un modèle de données versionné et sécurisé.

Travaux à réaliser

- Créer les tables nécessaires : profiles, goals, assessments, laws, lessons, exercises, programs, program_steps, daily_missions, mission_events, journal_entries, weekly_reviews et ai_runs.
- Créer des migrations reproductibles et des contraintes de clés étrangères.
- Activer les politiques RLS pour chaque table utilisateur.
- Séparer le contenu public publié du contenu brouillon d'administration.
- Ajouter des index pour les requêtes fréquentes.

Critères d'acceptation

- Les migrations peuvent être appliquées à un environnement neuf.
- Les utilisateurs ne peuvent ni lire ni modifier les données d'autrui.
- Les opérations administrateur sont protégées côté serveur.
- Les contraintes empêchent les états incohérents.

Tests obligatoires

- Tests automatisés RLS.
- Tests de migration sur une base de test.
- Tests des accès directs à l'API.

SPRINT 4 — Bibliothèque des 12 lois et gestion de contenu

Objectif : Créer un espace d'apprentissage administrable et sourcé.

Travaux à réaliser

- Créer les pages liste des lois, détail d'une loi, leçons, exemples, exercices, citations et références.
- Définir un schéma de contenu avec titre, résumé, texte, niveau, durée, tradition, sources et statut de publication.
- Ajouter recherche, filtres, favoris et progression de lecture si cela reste dans le périmètre MVP.
- Créer un back-office pour créer, éditer, prévisualiser, publier et archiver les contenus.
- Préparer une structure de contenu pour les 12 lois sans inventer des citations ou références.

Critères d'acceptation

- Chaque loi a une page et peut contenir plusieurs leçons.
- Les références sont conservées avec URL ou identifiant de source lorsque disponible.

- Les brouillons ne sont pas visibles aux utilisateurs ordinaires.
- Un administrateur peut publier et retirer un contenu.

Tests obligatoires

- Tests des permissions de publication.
- Tests de rendu des références et contenus longs.

Notes d'implémentation

- La formulation des 12 lois varie selon les traditions : documenter la tradition ou la source retenue.
- Faire relire les contenus spirituels avant publication.

SPRINT 5 — Objectifs et construction du parcours

Objectif : Transformer les objectifs en étapes réalisables sur une durée définie.

Travaux à réaliser

- Créer la gestion d'objectifs avec titre, description, priorité, indicateur, échéance et statut.
- Permettre les durées prises en charge : 21, 30, 60, 90 et 180 jours, avec possibilité d'extension contrôlée.
- Créer une logique de programme basée sur des règles explicites et une bibliothèque d'exercices.
- Générer des étapes avec numéro de jour, loi, leçon, mission, durée et objectif lié.
- Permettre à l'utilisateur de prévisualiser, confirmer, suspendre ou arrêter son parcours.
- Versionner le parcours lorsqu'une modification majeure est acceptée.

Critères d'acceptation

- Un parcours est lié à un utilisateur et à au moins un objectif.
- La charge quotidienne respecte le temps disponible déclaré.
- Les dates et numéros de jour sont cohérents avec le fuseau horaire.
- Le programme peut être repris après déconnexion.
- Aucun parcours n'est présenté comme garantissant le résultat souhaité.

Tests obligatoires

- Tests de durées, fuseaux horaires et dates limites.
- Tests des objectifs sans indicateur mesurable.
- Tests de modification et versionnement.

SPRINT 6 — Missions quotidiennes et journal

Objectif : Permettre à l'utilisateur d'agir et d'enregistrer ses résultats.

Travaux à réaliser

- Créer la page mission du jour avec consignes, durée estimée, loi associée et objectif associé.
- Gérer les états : à faire, commencé, terminé, reporté et ignoré.
- Enregistrer les événements de mission sans écraser l'historique.
- Créer le journal privé avec date, texte et questions de réflexion.

- Ajouter le bilan quotidien et la possibilité de reprendre une mission manquée.
- Éviter les pénalités et messages culpabilisants.

Critères d'acceptation

- La validation d'une mission est persistée en base.
- Le tableau de bord reflète les résultats enregistrés.
- Le journal est privé et soumis aux politiques d'accès.
- Les actions manquées peuvent être reprises sans dupliquer l'historique.

Tests obligatoires

- Tests de transitions de statut.
- Tests d'accès aux journaux.
- Tests de double clic et de soumission répétée.

SPRINT 7 — Agents IA de base et orchestrateur minimal

Objectif : Intégrer les quatre rôles IA prioritaires sans leur donner un accès illimité.

Travaux à réaliser

- Créer un service backend commun pour appels au modèle, timeout, retries limités, journalisation et gestion des erreurs.
- Implémenter les rôles Profil, Enseignement, Planificateur et Coach avec prompts versionnés.
- Fournir à chaque agent uniquement le contexte nécessaire et autorisé.
- Exiger des sorties structurées validées par schéma : résumé, proposition, références, avertissements et actions.
- Créer un orchestrateur minimal avec liste blanche des rôles et limite de coût/tours.
- Utiliser le corpus de leçons publié pour les réponses pédagogiques; afficher les sources utilisées.
- Prévoir un message de repli en cas d'indisponibilité du fournisseur IA.

Critères d'acceptation

- Les clés API sont uniquement côté serveur.
- Les sorties invalides sont rejetées ou régénérées un nombre limité de fois.
- L'agent Planificateur ne modifie pas directement la base sans validation backend.
- Le Coach n'accède pas automatiquement aux entrées du journal.
- Les erreurs fournisseur ne bloquent pas les fonctions non IA de l'application.

Tests obligatoires

- Tests avec sorties IA simulées.
- Tests de validation de schéma.
- Tests de timeout, quota et erreur fournisseur.
- Tests de fuite de données entre utilisateurs.

Notes d'implémentation

- Ne pas envoyer d'informations personnelles superflues au fournisseur.

- Ajouter limites de tokens, fréquence et coûts.

SPRINT 8 — Contrôle de qualité et sécurité IA

Objectif : Réduire les hallucinations, les réponses nuisibles et les actions non autorisées.

Travaux à réaliser

- Créer une étape de vérification des sources et des références lorsque le type de réponse l'exige.
- Définir des règles de refus, de reformulation et de transfert vers des ressources humaines appropriées.
- Distinguer les faits issus des données utilisateur, les interprétations et les suggestions.
- Empêcher l'injection d'instructions provenant de contenus utilisateur ou de documents non fiables.
- Ajouter des garde-fous contre les promesses de réussite, la culpabilisation et les affirmations spirituelles présentées comme des faits scientifiques.
- Journaliser les métadonnées des exécutions IA sans stocker inutilement les prompts privés complets.

Critères d'acceptation

- Les agents ne peuvent appeler que les outils explicitement autorisés.
- Les références non vérifiées ne sont pas présentées comme certaines.
- Les contenus générés ne sont pas publiés automatiquement.
- Les réponses risquées sont bloquées ou orientées vers une aide adaptée.

Tests obligatoires

- Tests adversariaux de prompt injection.
- Tests de citations inventées.
- Tests des limites d'autorisation et des réponses sensibles.

SPRINT 9 — Progression, bilans et adaptation

Objectif : Transformer l'historique des actions en ajustements utiles.

Travaux à réaliser

- Calculer des indicateurs déterministes : missions réalisées, fréquence, temps consacré et jalons atteints.
- Créer les vues semaine, mois et parcours complet.
- Ajouter un agent Progression qui prépare un résumé à partir des indicateurs réels.
- Proposer des ajustements : réduire une charge, fractionner une action, réordonner une étape ou maintenir le programme.
- Demander une confirmation pour les changements majeurs.
- Afficher les limites des statistiques et éviter les scores présentés comme mesure absolue de la valeur personnelle.

Critères d'acceptation

- Les graphiques utilisent des données réellement enregistrées.
- Les calculs de base sont effectués par du code déterministe, pas inventés par l'IA.
- L'utilisateur peut comprendre pourquoi un ajustement est proposé.

- Les changements de programme sont traçables et réversibles lorsque possible.

Tests obligatoires

- Tests des calculs et périodes.
- Tests de données manquantes.
- Tests de validation utilisateur avant changements majeurs.

SPRINT 10 — Rappels, tâches planifiées et notifications

Objectif : Automatiser les moments de suivi sans perturber l'utilisateur.

Travaux à réaliser

- Créer des tâches planifiées idempotentes pour missions quotidiennes et bilans hebdomadaires.
- Respecter le fuseau horaire, les préférences et les périodes silencieuses.
- Ajouter les préférences de notifications et un mécanisme de désinscription.
- Envoyer des rappels non culpabilisants et limiter leur fréquence.
- Prévoir un système de reprise si une tâche planifiée échoue.

Critères d'acceptation

- Une tâche planifiée ne crée pas de doublons si elle est rejouée.
- Les notifications respectent les préférences de l'utilisateur.
- Les erreurs sont journalisées et peuvent être réessayées sans effets indésirables.
- L'utilisateur peut désactiver les notifications.

Tests obligatoires

- Tests de fuseau horaire et heure d'été/hiver.
- Tests d'idempotence.
- Tests de désinscription et de fréquence.

SPRINT 11 — Administration, observabilité et coûts

Objectif : Donner au propriétaire les outils nécessaires pour gérer le produit.

Travaux à réaliser

- Créer un espace admin protégé pour contenus, utilisateurs et statut des tâches.
- Afficher des métriques agrégées : parcours actifs, missions complétées, erreurs et latence IA.
- Ajouter limites d'usage, alertes de coût et quotas par utilisateur.
- Créer des journaux techniques avec masquage des données personnelles.
- Permettre l'archivage et la correction des contenus publiés.

Critères d'acceptation

- Les routes administrateur sont protégées côté serveur.
- Les métriques ne révèlent pas les journaux personnels.
- Les quotas et limites sont appliqués côté serveur.

- Les erreurs peuvent être diagnostiquées sans exposer de secrets.

Tests obligatoires

- Tests d'accès admin.
- Tests de quota.
- Vérification des logs pour données personnelles et secrets.

SPRINT 12 — Confidentialité, export et suppression

Objectif : Donner aux utilisateurs le contrôle de leurs données.

Travaux à réaliser

- Créer une page de confidentialité expliquant les données stockées et les usages IA.
- Permettre l'export des données utilisateur dans un format exploitable.
- Permettre la suppression du compte et des données associées selon la politique définie.
- Définir une durée de conservation des journaux techniques et des exécutions IA.
- Documenter les fournisseurs tiers et les données éventuellement transmises.
- Prévoir le consentement nécessaire et les réglages de confidentialité.

Critères d'acceptation

- L'utilisateur peut demander l'export de ses données.
- La suppression est authentifiée, confirmée et traitée de manière cohérente.
- Les données personnelles ne sont pas conservées sans raison définie.
- Les intégrations et usages IA sont expliqués clairement.

Tests obligatoires

- Tests d'export et de suppression.
- Tests de conservation et de révocation d'accès.

Notes d'implémentation

- Vérifier les obligations légales applicables avant mise en production, notamment le RGPD.

SPRINT 13 — Tests de bout en bout, accessibilité et mise en production

Objectif : Valider le parcours complet et déployer une version fiable.

Travaux à réaliser

- Créer des tests end-to-end du parcours : inscription, questionnaire, objectif, programme, mission, journal et bilan.
- Vérifier les performances, la navigation clavier, les contrastes et les erreurs d'interface.
- Créer les environnements développement, préproduction et production.
- Configurer les variables d'environnement dans l'hébergeur, jamais dans le dépôt.
- Mettre en place des sauvegardes et documenter la procédure de restauration.

- Rédiger le README de déploiement, le guide utilisateur et le guide d'administration.

Critères d'acceptation

- Les tests critiques passent dans le pipeline CI.
- Le build de production est réussi.
- Aucun secret n'est inclus dans le dépôt ou les artefacts publics.
- La procédure de déploiement et de retour arrière est documentée.
- Un test de restauration de sauvegarde est réalisé avant le lancement.

Tests obligatoires

- Tests end-to-end sur environnement de préproduction.
- Audit de dépendances et vérification des secrets.
- Tests de restauration et de retour arrière.

8. Définition de terminé pour chaque sprint

- Le code est cohérent avec l'architecture existante et les conventions du dépôt.
- Les tests pertinents sont ajoutés ou mis à jour et exécutés.
- Le build ou les contrôles statiques passent, ou les échecs sont documentés.
- Les migrations, variables d'environnement et changements de configuration sont documentés.
- Aucun secret ni donnée réelle d'utilisateur n'est ajouté au dépôt.
- Le README ou la documentation technique est mis à jour si nécessaire.
- Un résumé précise les fichiers modifiés, les tests exécutés, les résultats et les limites restantes.
- Un commit local est préparé après validation; push vers GitHub uniquement sur autorisation.

9. Mise sur GitHub : procédure sécurisée

GitHub sert à versionner le code, suivre les modifications et collaborer. Le dépôt peut être privé pendant le développement, ce qui est recommandé tant que le produit et la configuration ne sont pas prêts à être publics.

A. Vérifier le projet en local

```
git status
git branch --show-current
git log -5 --oneline
```

Si Git n'est pas encore initialisé, vérifier d'abord le dossier courant et le contenu du projet. Ne pas exécuter git init à l'aveugle dans un dossier parent qui contient d'autres projets.

B. Créer le dépôt GitHub

1. Se connecter à GitHub et créer un nouveau dépôt nommé spirit12 (ou un nom disponible).
2. Choisir Private pour commencer.
3. Si le projet existe déjà en local, ne pas cocher l'ajout automatique d'un README, d'une licence ou d'un .gitignore qui pourrait provoquer un conflit initial.
4. Copier l'URL HTTPS ou SSH du dépôt créé.

C. Préparer les fichiers avant le premier push

Vérifier que .gitignore exclut au minimum les éléments suivants selon la stack :

```
node_modules/
.next/
out/
dist/
build/
coverage/
.env
.env.*
!.env.example
*.log
.DS_Store
Thumbs.db
```

Ne jamais ignorer le fichier .env.example : il doit contenir uniquement des noms de variables et des valeurs factices. Vérifier également qu'aucun export de base de données, fichier client, journal privé ou clé API ne sera ajouté.

D. Premier commit et liaison au dépôt distant

```
git add .
git status
git diff --cached
git commit -m "chore: initial SPIRIT 12 project"
git branch -M main
git remote add origin https://github.com/USERNAME/spirit12.git
git push -u origin main
```

Remplacer USERNAME et l'URL par l'adresse exacte du dépôt créé. Si origin existe déjà, utiliser git remote -v avant toute modification. Ne pas refaire git remote add origin si le remote existe.

E. Flux de travail conseillé pour chaque sprint

```
git checkout -b feature/sprint-01-foundations
# Implémenter le sprint et exécuter les tests
git status
git diff
git add <fichiers-validés>
git commit -m "feat: implement sprint 01 foundations"
git push -u origin feature/sprint-01-foundations
# Ouvrir une Pull Request sur GitHub, vérifier la CI, puis fusionner
```

Si tu es seul sur le projet, tu peux travailler directement avec des branches et fusionner après vérification. Les Pull Requests restent utiles pour relire les changements et garder une trace des décisions.

10. GitHub Actions / intégration continue

Configurer une action CI après avoir identifié les scripts réels du dépôt. Exemple indicatif pour un projet Node.js avec npm; OpenCode doit adapter les commandes et la version de Node à package.json et au lockfile.

```
name: CI
on:
pull_request:
push:
branches: [main]
jobs:
test:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: actions/setup-node@v4
with:
node-version: 22
cache: npm
- run: npm ci
```

```
- run: npm run lint --if-present
- run: npm run typecheck --if-present
- run: npm test --if-present
- run: npm run build
```

Les secrets de production ne doivent pas être nécessaires aux tests unitaires. Si des tests d'intégration ont besoin de secrets, utiliser des GitHub Actions Secrets limités et des services de test isolés. Ne jamais écrire les valeurs secrètes dans les logs.

11. README minimal attendu sur GitHub

```
# SPIRIT 12

Application web d'enseignement spirituel et d'accompagnement personnalisé.

## Fonctionnalités
- Parcours personnalisé et objectifs
- Bibliothèque des 12 lois
- Missions quotidiennes et journal privé
- Accompagnement IA encadré
- Suivi de progression

## Prérequis
- Node.js compatible avec le projet
- Gestionnaire de paquets défini par le lockfile
- Projet Supabase ou backend configuré
- Clé API du fournisseur IA (côté serveur uniquement)

## Installation
1. Copier `.env.example` vers `.env.local`.
2. Renseigner les variables locales sans les publier.
3. Installer les dépendances avec le gestionnaire du projet.
4. Appliquer les migrations de base de données.
5. Lancer le serveur de développement.

## Sécurité
Ne jamais publier de clés API, secrets, données personnelles ou journaux privés.

## Développement
Voir `docs/` pour les sprints, l'architecture et les procédures.
```

12. Variables d'environnement et secrets

- Conserver les secrets dans les variables d'environnement locales ou le gestionnaire de secrets de l'hébergeur.
- Pour Supabase, ne jamais exposer la clé service-role au navigateur. Les clés publiques doivent rester soumises aux règles RLS.
- Pour le fournisseur IA, appeler l'API uniquement depuis le serveur.
- Fournir un .env.example avec des valeurs fictives et des commentaires.
- Si un secret est publié accidentellement, le révoquer immédiatement, en générer un nouveau et vérifier l'historique Git; supprimer le fichier dans un commit ne suffit pas à invalider le secret.

13. Prompt GitHub à donner à OpenCode

```
Prépare le projet SPIRIT 12 pour GitHub de façon sécurisée.
1. Inspecte l'état Git, la branche, les remotes et les fichiers suivis.
2. Ne publie rien et ne pousse aucun commit sans me demander confirmation.
3. Vérifie .gitignore et repère les secrets, fichiers .env, données utilisateur, exports et
journaux privés.
4. Crée ou améliore README.md et .env.example avec des valeurs factices seulement.
5. Si Git n'est pas initialisé, propose les commandes adaptées au dossier du projet sans les
exécuter avant confirmation.
6. Si un remote existe, ne le remplace pas. Si aucun remote n'existe, demande-moi l'URL exacte du
dépôt GitHub.
7. Lance les tests pertinents et affiche git status et git diff avant tout commit.
8. Propose un commit descriptif et un push uniquement après mon approbation explicite.
Ne divulgue aucune valeur secrète dans le rapport.
```

14. Checklist avant ouverture du dépôt

- Le dépôt est privé au début, sauf décision contraire.
- .gitignore est adapté à la stack.
- Aucun fichier .env réel ou token n'est suivi par Git.
- Aucune donnée personnelle, entrée de journal ou export de base n'est présent.
- README et .env.example sont présents.
- Les tests et le build passent.
- La licence est choisie avant de rendre le dépôt public.
- Les règles de branche et la CI sont configurées.
- Les secrets sont configurés dans l'hébergeur ou les paramètres sécurisés de CI.
- Les droits d'accès au dépôt sont limités aux personnes nécessaires.

15. Ordre de réalisation conseillé

1. Ouvrir le dépôt dans OpenCode et exécuter uniquement le Sprint 0 (audit).
2. Valider l'architecture et le périmètre avant toute modification importante.
3. Réaliser les sprints 1 à 6 pour obtenir une application utilisable sans IA avancée.
4. Ajouter les agents de base au Sprint 7, puis les contrôles de sécurité.
5. Ajouter l'analyse de progression, les rappels et l'administration.
6. Effectuer les tests de confidentialité, export, suppression et sécurité.
7. Préparer le dépôt GitHub privé et la CI, puis déployer d'abord en préproduction.

Livrable attendu à la fin de chaque sprint : code fonctionnel, tests, documentation mise à jour, résumé des modifications et proposition de commit. Aucun sprint ne doit être considéré comme terminé uniquement parce que le code a été généré.